

zadania_lab_14

Metody Monte Carlo — zadania

Wprowadzenie

Metody Monte Carlo polegają na wykorzystaniu losowania do przybliżonego rozwiązania problemu matematycznego.

Zamiast liczyć wynik dokładnie, wykonujemy dużo losowych prób, a następnie uśredniamy wynik.

Ogólny schemat wygląda tak:

1. określamy obszar, z którego losujemy punkty,
2. losujemy dużą liczbę punktów,
3. dla każdego punktu wykonujemy obliczenie,
4. na podstawie średniej albo proporcji punktów otrzymujemy wynik przybliżony.

Im większa liczba punktów N , tym wynik zwykle jest dokładniejszy.

Na wykładzie podano, że błąd metody Monte Carlo maleje w przybliżeniu jak:

$$O\left(\frac{1}{\sqrt{N}}\right)$$

czyli jeśli chcemy zmniejszyć błąd 10 razy, to trzeba zwiększyć liczbę próbek około 100 razy.

Zadanie 1 — całkowanie metodą Monte Carlo

Treść zadania:

Wykorzystaj metodę Monte Carlo do obliczenia przybliżonej wartości całek:

a)

$$\int_0^1 x^2 dx$$

b)

$$\int_e^{e^2} \frac{1}{x} dx$$

c)

$$\iint_D (\cos x + y + 1) \, dx dy \quad D = [0, 2] \times [-\pi, \pi]$$

Oszacuj liczbę punktów potrzebnych do uzyskania dokładności do 2 cyfr po przecinku.

Metoda Crude Monte Carlo

Dla całki jednowymiarowej:

$$I = \int_a^b f(x) \, dx$$

losujemy N punktów:

$$x_1, x_2, \dots, x_N \in [a, b]$$

a następnie liczymy:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

Czyli w praktyce:

1. losujemy dużo punktów x z przedziału $[a, b]$,
 2. liczymy wartości funkcji $f(x)$,
 3. obliczamy średnią tych wartości,
 4. mnożymy przez długość przedziału $b - a$.
-

Całka podwójna

Dla całki po prostokącie:

$$D = [a, b] \times [c, d]$$

losujemy punkty:

$$(x_i, y_i)$$

i korzystamy ze wzoru:

$$I \approx \frac{(b-a)(d-c)}{N} \sum_{i=1}^N f(x_i, y_i)$$

czyli średnią wartość funkcji mnożymy przez pole obszaru.

Wartości dokładne do porównania

a)

$$\int_0^1 x^2, dx = \frac{x^3}{3} \Big|_0^1$$

$$\int_0^1 x^2, dx = \frac{1}{3}$$

b)

$$\int_e^{e^2} \frac{1}{x}, dx = \ln x \Big|_e^{e^2}$$

$$\int_e^{e^2} \frac{1}{x}, dx = \ln(e^2) - \ln(e) = 2 - 1 = 1$$

c)

$$\iint_D (\cos x + y + 1), dx dy$$

gdzie:

$$D = [0, 2] \times [-\pi, \pi]$$

Najpierw zauważamy, że składnik y znika, ponieważ całkujemy go po symetrycznym przedziale:

$$\int_{-\pi}^{\pi} y, dy = 0$$

Zostaje:

$$\begin{aligned} & \int_0^2 \int_{-\pi}^{\pi} (\cos x + 1) dy dx \\ &= \int_0^2 2\pi(\cos x + 1), dx \\ &= 2\pi(\sin 2 + 2) \end{aligned}$$

Czyli wartość dokładna wynosi:

$$2\pi(\sin 2 + 2)$$

Jak oszacować liczbę punktów?

Chcemy dokładność do 2 cyfr po przecinku, więc przyjmujemy błąd około:

$$\varepsilon = 0.005$$

Ponieważ błąd Monte Carlo zachowuje się mniej więcej jak:

$$\frac{1}{\sqrt{N}}$$

to liczba punktów potrzebna do uzyskania dokładności zależy od rozrzutu wartości funkcji.

W programie robimy to praktycznie:

1. losujemy próbnie pewną liczbę punktów,
2. liczymy odchylenie standardowe wartości funkcji,
3. szacujemy potrzebne N .

N bierze się z zależności:

$$\text{błąd} \approx \frac{(b-a)\sigma}{\sqrt{N}}$$

gdzie:

$$\sigma$$

to odchylenie standardowe wartości $f(x)$, a N to liczba próbek.

Jeżeli chcesz, żeby błąd był mniejszy niż zadana dokładność:

$$\varepsilon$$

to zapisujesz:

$$\frac{(b-a)\sigma}{\sqrt{N}} \leq \varepsilon$$

Przekształcamy:

$$\sqrt{N} \geq \frac{(b-a)\sigma}{\varepsilon}$$

Podnosimy do kwadratu:

$$N \geq \left(\frac{(b-a)\sigma}{\varepsilon} \right)^2$$

I to jest dokładnie w kodzie:

```
N = ((b - a) * odchylenie / dokladnosc) ** 2
```

PYTHON

Czyli ten wzór może nie być w wykładzie zapisany dosłownie jako gotowa funkcja do **N**, ale powinien wynikać z fragmentu o tym, że błąd Monte Carlo maleje jak:

$$\frac{1}{\sqrt{N}}$$

albo z odchylenia standardowego średniej:

$$\frac{\sigma}{\sqrt{N}}$$

Wzór na liczbę próbek wynika z oszacowania błędu metody Monte Carlo. Ponieważ błąd jest proporcjonalny do $\frac{1}{\sqrt{N}}$, to dla zadanej dokładności ε można oszacować liczbę próbek jako:

$$N \approx \left(\frac{(b-a)\sigma}{\varepsilon} \right)^2$$

gdzie σ jest odchyleniem standardowym wartości funkcji $f(x)$ w losowanych punktach.

Kod do zadania 1

```
import random # importujemy moduł random, bo potrzebujemy losowania liczb
import math # importujemy moduł math, bo potrzebujemy liczby e, pi oraz funkcji
cos i sin

print("----- ZADANIE 1 -----") # wypisujemy nagłówek zadania

def monte_carlo_1d(f, a, b, N): # funkcja oblicza całkę jednowymiarową metodą
    Monte Carlo
    suma = 0.0 # tworzymy zmienną, w której będziemy sumować wartości funkcji

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(a, b) # losujemy punkt x z przedziału [a,b]
        suma += f(x) # dodajemy wartość funkcji w wylosowanym punkcie

    return ((b - a) / N) * suma # zwracamy przybliżenie całki według wzoru z
    wykładu

def monte_carlo_2d(f, ax, bx, ay, by, N): # funkcja oblicza całkę podwójną metodą
    Monte Carlo
    suma = 0.0 # tworzymy zmienną na sumę wartości funkcji

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(ax, bx) # losujemy współrzędną x
        y = random.uniform(ay, by) # losujemy współrzędną y
        suma += f(x, y) # dodajemy wartość funkcji w punkcie (x,y)

    pole_obszaru = (bx - ax) * (by - ay) # obliczamy pole prostokąta, po którym
    całkujemy

    return (pole_obszaru * suma) / N # zwracamy przybliżenie całki podwójnej

def oszacuj_N_1d(f, a, b, dokladnosc=0.005, N_probne=10000): # funkcja szacuje
    potrzebną liczbę punktów dla całki 1D
    wartosci = [] # lista na wartości funkcji w losowych punktach

    for i in range(N_probne): # wykonujemy próbne losowania
        x = random.uniform(a, b) # losujemy punkt x
        wartosci.append(f(x)) # zapisujemy wartość funkcji

    srednia = sum(wartosci) / N_probne # liczymy średnią wartości funkcji

    wariancja = 0.0 # zmienna na wariancję
    for wartosc in wartosci: # przechodzimy po wszystkich wartościach
        wariancja += (wartosc - srednia) ** 2 # dodajemy kwadrat różnicy od
    średniej

    wariancja = wariancja / (N_probne - 1) # kończymy obliczanie wariancji
```

```

    odchylenie = math.sqrt(wariancja) # liczymy odchylenie standardowe

    N = ((b - a) * odchylenie / dokladnosc) ** 2 # szacujemy liczbę punktów z
    zależności błędu

    return math.ceil(N) # zaokrąglamy w górę do liczby całkowitej

def oszacuj_N_2d(f, ax, bx, ay, by, dokladnosc=0.005, N_probne=10000): # funkcja
    szacuje potrzebne N dla całki 2D
    wartosci = [] # lista na wartości funkcji

    for i in range(N_probne): # wykonujemy próbne losowania
        x = random.uniform(ax, bx) # losujemy x
        y = random.uniform(ay, by) # losujemy y
        wartosci.append(f(x, y)) # zapisujemy wartość funkcji

    srednia = sum(wartosci) / N_probne # liczymy średnią

    wariancja = 0.0 # zmienna na wariancję
    for wartosc in wartosci: # przechodzimy po wartościach
        wariancja += (wartosc - srednia) ** 2 # dodajemy kwadrat odchylenia od
    średniej

    wariancja = wariancja / (N_probne - 1) # liczymy wariancję
    odchylenie = math.sqrt(wariancja) # liczymy odchylenie standardowe

    pole_obszaru = (bx - ax) * (by - ay) # obliczamy pole obszaru całkowania

    N = (pole_obszaru * odchylenie / dokladnosc) ** 2 # szacujemy potrzebną
    liczbę punktów

    return math.ceil(N) # zwracamy liczbę punktów zaokrągloną w górę

def f1(x): # funkcja z podpunktu a)
    return x ** 2 # f(x)=x^2

def f2(x): # funkcja z podpunktu b)
    return 1 / x # f(x)=1/x

def f3(x, y): # funkcja z podpunktu c)
    return math.cos(x) + y + 1 # f(x,y)=cos(x)+y+1

N = 100000 # liczba losowanych punktów do właściwego obliczenia całek

wynik_a = monte_carlo_1d(f1, 0, 1, N) # obliczamy całkę a) metodą Monte Carlo
dokladny_a = 1 / 3 # dokładna wartość całki a)

```

```

wynik_b = monte_carlo_1d(f2, math.e, math.e ** 2, N) # obliczamy całkę b)
dokladny_b = 1 # dokładna wartość całki b)

wynik_c = monte_carlo_2d(f3, 0, 2, -math.pi, math.pi, N) # obliczamy całkę c)
dokladny_c = 2 * math.pi * (math.sin(2) + 2) # dokładna wartość całki c)

print("\na) całka od 0 do 1 z x^2 dx") # wypisujemy opis całki a)
print("Wynik Monte Carlo:", wynik_a) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladny_a) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladny_a - wynik_a)) # wypisujemy błąd
bezwzględny

print("\nb) całka od e do e^2 z 1/x dx") # wypisujemy opis całki b)
print("Wynik Monte Carlo:", wynik_b) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladny_b) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladny_b - wynik_b)) # wypisujemy błąd
bezwzględny

print("\nc) całka podwójna po D = [0,2] x [-pi,pi]") # wypisujemy opis całki c)
print("Wynik Monte Carlo:", wynik_c) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladny_c) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladny_c - wynik_c)) # wypisujemy błąd
bezwzględny

print("\nOszacowanie liczby punktów dla dokładności do 2 cyfr po przecinku:") #
nagłówek oszacowania N

N_a = oszacuj_N_1d(f1, 0, 1) # szacujemy liczbę punktów dla całki a)
N_b = oszacuj_N_1d(f2, math.e, math.e ** 2) # szacujemy liczbę punktów dla całki
b)
N_c = oszacuj_N_2d(f3, 0, 2, -math.pi, math.pi) # szacujemy liczbę punktów dla
całki c)

print("a) potrzebne N ≈", N_a) # wypisujemy szacowane N dla a)
print("b) potrzebne N ≈", N_b) # wypisujemy szacowane N dla b)
print("c) potrzebne N ≈", N_c) # wypisujemy szacowane N dla c)

```

Wnioski do zadania 1

Metoda Monte Carlo pozwala przybliżyć wartość całki przez losowanie punktów i liczenie średniej wartości funkcji.

Dla całek jednowymiarowych używamy wzoru:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

Dla całki podwójnej po prostokącie używamy wzoru:

$$I \approx \frac{P_D}{N} \sum_{i=1}^N f(x_i, y_i)$$

gdzie P_D oznacza pole obszaru całkowania.

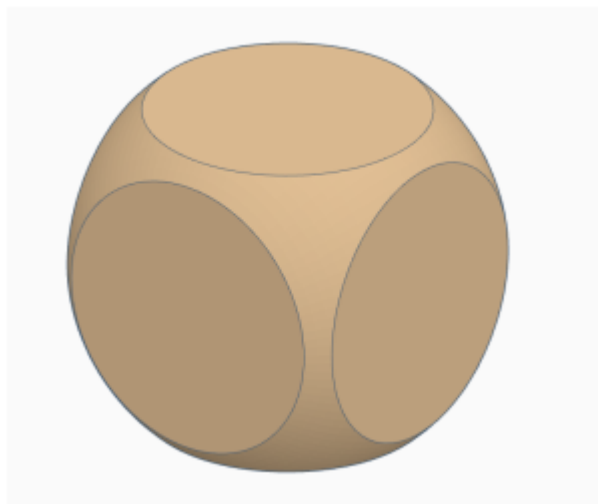
Wynik jest losowy, dlatego po każdym uruchomieniu programu może być trochę inny. Przy większym N wynik powinien być bliższy wartości dokładnej.

Zadanie 2 — metoda akceptacji i odrzuceń

Treść zadania:

Stosując metodę akceptacji i odrzuceń oblicz:

- objętość kuli jednostkowej,
- objętość części wspólnej sześcianu i kuli, przy czym stosunek promienia kuli do długości boku sześcianu wynosi 2 : 3.



Na czym polega metoda akceptacji i odrzuceń?

Metoda akceptacji–odrzucań polega na losowaniu punktów z prostego obszaru, a następnie sprawdzaniu, czy wylosowany punkt należy do badanego obszaru.

Jeżeli:

- N — liczba wszystkich wylosowanych punktów,
- k — liczba punktów zaakceptowanych,

- V_{obszaru} — objętość obszaru, z którego losujemy,

to szukana objętość jest przybliżona wzorem:

$$V \approx \frac{k}{N} V_{\text{obszaru}}$$

To jest odpowiednik wzoru ze slajdów:

$$I \approx \frac{k}{N} P_{\text{prostokąta}}$$

Ponieważ:

$$P_{\text{prostokąta}} = (b - a)M$$

otrzymujemy:

$$I \approx \frac{k}{N} (b - a)M$$

tylko zamiast pola prostokąta mamy objętość sześcianu.

Zadanie 2a — objętość kuli jednostkowej

Kula jednostkowa ma promień:

$$r = 1$$

Losujemy punkty z sześcianu:

$$[-1, 1] \times [-1, 1] \times [-1, 1]$$

Bo ma środek w punkcie:

$$(0, 0, 0)$$

Czyli kula obejmuje punkty oddalone od środka maksymalnie o 1.

W każdej osi wygląda to tak:

x od -1 do 1 y od -1 do 1 z od -1 do 1

Gdyby kula miała promień $r = 2$, wtedy losowalibyśmy z:

$$[-2, 2] \times [-2, 2] \times [-2, 2]$$

Objętość tego sześcianu wynosi:

$$V_{\text{sześcianu}} = 2 \cdot 2 \cdot 2 = 8$$

Uniwersalna metoda na liczenie objętości sześcianu o $r = 1$

```

x_min = -1
x_max = 1

y_min = -1
y_max = 1

z_min = -1
z_max = 1

objetosc = (x_max - x_min) * (y_max - y_min) * (z_max - z_min)

print("Objętość =", objetosc)

```

Punkt należy do kuli, jeżeli spełnia warunek:

$$x^2 + y^2 + z^2 \leq 1$$

Liczba punktów spełniających ten warunek to k .

Wtedy objętość kuli przybliżamy wzorem:

$$V_{\text{kuli}} \approx 8 \cdot \frac{k}{N}$$

Wartość dokładna objętości kuli jednostkowej wynosi:

$$V = \frac{4}{3}\pi$$

Zadanie 2b — część wspólna sześcianu i kuli

Z treści zadania:

$$r : bok = 2 : 3$$

Przyjmujemy więc:

$$r = 2$$

oraz:

$$bok = 3$$

Sześcian ustawiamy symetrycznie względem początku układu współrzędnych, dlatego losujemy punkty z przedziałów (w kodzie to zmienna *polowa_boku*):

$$[-1.5, 1.5] \times [-1.5, 1.5] \times [-1.5, 1.5]$$

Objętość sześcianu wynosi:

$$V_{\text{sześcianu}} = 3^3 = 27$$

Punkt należy do kuli o promieniu (2), jeśli:

$$x^2 + y^2 + z^2 \leq 2^2$$

czyli:

$$x^2 + y^2 + z^2 \leq 4$$

Ponieważ losujemy punkty już ze środka sześcianu, wystarczy sprawdzić tylko warunek kuli. Punkty spełniające warunek są częścią wspólną sześcianu i kuli.

Objętość części wspólnej przybliżamy wzorem:

$$V \approx 27 \cdot \frac{k}{N}$$

Kod do zadania 2

```

import random # importujemy moduł random do losowania punktów
import math # importujemy math, ponieważ potrzebujemy liczby pi

print("----- ZADANIE 2 -----") # wypisujemy nagłówek zadania

# a)
def objetosc_kuli_jednostkowej(N): # funkcja oblicza objętość kuli jednostkowej
    metodą akceptacji i odrzuceń
    zaakceptowane = 0 # licznik punktów, które trafiły do kuli

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(-1, 1) # losujemy współrzędną x z przedziału [-1,1]
        y = random.uniform(-1, 1) # losujemy współrzędną y z przedziału [-1,1]
        z = random.uniform(-1, 1) # losujemy współrzędną z z przedziału [-1,1]

        if x ** 2 + y ** 2 + z ** 2 <= 1: # sprawdzamy, czy punkt leży wewnątrz
            kuli jednostkowej
            zaakceptowane += 1 # jeśli tak, zwiększamy licznik zaakceptowanych
            punktów

    objetosc_szescianu = 2 ** 3 # objętość sześcianu [-1,1]^3 wynosi 8

    return objetosc_szescianu * zaakceptowane / N # zwracamy przybliżoną objętość
    kuli

# b)
def objetosc_wspolna_szescianu_i_kuli(N): # funkcja oblicza objętość części
    wspólnej sześcianu i kuli
    r = 2 # promień kuli zgodnie ze stosunkiem 2:3
    bok = 3 # bok sześcianu zgodnie ze stosunkiem 2:3

    polowa_boku = bok / 2 # połowa boku sześcianu, czyli 1.5

    zaakceptowane = 0 # licznik punktów, które znajdują się jednocześnie w
    sześcianie i kuli

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(-polowa_boku, polowa_boku) # losujemy x z przedziału
        [-1.5,1.5]
        y = random.uniform(-polowa_boku, polowa_boku) # losujemy y z przedziału
        [-1.5,1.5]
        z = random.uniform(-polowa_boku, polowa_boku) # losujemy z z przedziału
        [-1.5,1.5]

        if x ** 2 + y ** 2 + z ** 2 <= r ** 2: # sprawdzamy, czy punkt należy do
            kuli o promieniu 2
            zaakceptowane += 1 # jeśli tak, punkt należy do części wspólnej

    objetosc_szescianu = bok ** 3 # objętość sześcianu o boku 3 wynosi 27

```

```

    return objetosc_szescianu * zaakceptowane / N # zwracamy przybliżoną objętość
    części wspólnej

N = 200000 # liczba losowanych punktów

wynik_kula = objetosc_kuli_jednostkowej(N) # obliczamy objętość kuli jednostkowej
dokladna_kula = 4 / 3 * math.pi # dokładna objętość kuli jednostkowej

wynik_wspolna = objetosc_wspolna_szescianu_i_kuli(N) # obliczamy objętość części
wspólnej sześcianu i kuli

print("\na) Objętość kuli jednostkowej") # nagłówek podpunktu a)
print("Wynik Monte Carlo:", wynik_kula) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladna_kula) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladna_kula - wynik_kula)) # wypisujemy błąd
bezwzględny

print("\nb) Objętość części wspólnej sześcianu i kuli") # nagłówek podpunktu b)
print("Stosunek promienia kuli do boku sześcianu: 2:3") # wypisujemy stosunek z
zadania
print("Przyjmujemy r = 2 oraz bok = 3") # wypisujemy przyjęte wartości
print("Wynik Monte Carlo:", wynik_wspolna) # wypisujemy wynik Monte Carlo

```

Wnioski do zadania 2

W zadaniu 2 użyto metody akceptacji–odrzuceń.

Dla kuli jednostkowej losujemy punkty z sześcianu:

$$[-1, 1]^3$$

i sprawdzamy warunek:

$$x^2 + y^2 + z^2 \leq 1$$

Dla części wspólnej sześcianu i kuli losujemy punkty z sześcianu o boku 3 i sprawdzamy, które z nich znajdują się również w kuli o promieniu 2.

W obu przypadkach wynik ma postać:

$$V \approx V_{\text{obszaru}} \cdot \frac{k}{N}$$

gdzie k oznacza liczbę zaakceptowanych punktów.

Metoda jest prosta, ale wynik jest przybliżony i zależy od liczby losowań. Im większe N , tym wynik powinien być dokładniejszy.